



DAAN862: ANALYTICS PROGRAMMING IN PYTHON

Lesson 5: Data Wrangling with Pandas

Lesson 5.2: Merge DataSets

There are two main functions in pandas to combine different datasets: ***pandas.merge*** & ***pandas.concat***

(click the titles to learn more)

`pandas.merge`

See Table 5.2.1 for the arguments of the *merge* function:

Table 5.1.2: Arguments of the Merge Function

Argument	Description
left	DataFrame to be merged on the left side
right	DataFrame to be merged on the right side
how	One of 'inner', 'outer', 'left' or 'right'. 'inner' by default
on	Column names to join on. Must be found in both DataFrame objects
left on	Columns in left DataFrame to use as join keys
right on	Analogous to left_on for left DataFrame
left index	left_index Use row index in left as its join key (or keys, if a MultiIndex)
right index	Analogous to left_index
	Sort merged data lexicographically by join keys; True by default. Disable to get

sort	better performance in some cases on large datasets
suffixes	Tuple of string values to append to column names in case of overlap; defaults to ('_x', '_y'). For example, if 'data' in both DataFrame objects, would appear as 'data_x' and 'data_y' in result
copy	If False, avoid copying data into resulting data structure in some exceptional cases. By default always copies

(click the tabs to learn more about pandas.merge)

Merge On Key

Merge On Index

Merge on Key

Merge function can contribute datasets using one or more keys.

Figure 5.17 shows two DataFrames as examples:

```
In [28]: df1 = pd.DataFrame({'key': list('aabccde'),
...:                        'value1': range(7)})
...: df1
Out[28]:
   key  value1
0    a        0
1    a        1
2    b        2
3    c        3
4    c        4
5    d        5
6    e        6

In [29]: df2 = pd.DataFrame({'key': list('ace'),
...:                        'value2': range(4, 7)})
...: df2
Out[29]:
   key  value2
0    a        4
1    c        5
2    e        6
```

Fig 5.17 (click to enlarge)

Merge df1 and df2 is an example of a many-to-one situation. The 'key' column of df1 has multiple rows with the values of a and c, while the 'key' column of df2 has unique values. Figure 5.18 uses *merge* functions:

```
In [30]: pd.merge(df1, df2)
```

```
Out[30]:
```

	key	value1	value2
0	a	0	4
1	a	1	4
2	c	3	5
3	c	4	5
4	e	6	6

Fig 5.18 (click to enlarge)

Since we didn't set which column to join on, *merge* function will use the overlapping column as the keys, and by default it will use an inner join method. That's why the d and be values are missing in Figure 5.18

Let's try to specify the columns clearly which will return the same results as shown in Figure 5.19:

```
In [31]: pd.merge(df1, df2, on = 'key')
```

```
Out[31]:
```

	key	value1	value2
0	a	0	4
1	a	1	4
2	c	3	5
3	c	4	5
4	e	6	6

Fig 5.19 (click to enlarge)

Sometimes, the column names might be different in two datasets. In that case, you can specify them separately by using *left on* and *right on* arguments as shown in Figure 5.20:

```
In [33]: df1.columns = ['key1', 'value1'] # reset column names
```

```

...: df1
Out[33]:
   key1  value1
0     a       0
1     a       1
2     b       2
3     c       3
4     c       4
5     d       5
6     e       6

In [34]: df2.columns = ['key2', 'value2']  # reset column names
...: df2
Out[34]:
   key2  value2
0     a       4
1     c       5
2     e       6

In [35]: pd.merge(df1, df2, left_on = 'key1', right_on = 'key2')
Out[35]:
   key1  value1 key2  value2
0     a       0     a       4
1     a       1     a       4
2     c       3     c       5
3     c       4     c       5
4     e       6     e       6

```

Fig 5.20 (click to enlarge)

Instead of using default *how* = 'inner', there are other possible options such as 'left', 'right', and 'outer'.

Let's try to use *how* = 'outer', which will give the union of the keys as shown in Figure 5.21:

```

In [36]: pd.merge(df1, df2, left_on = 'key1', right_on = 'key2', how = 'outer')
Out[36]:
   key1  value1 key2  value2
0     a       0     a     4.0
1     a       1     a     4.0
2     b       2  NaN     NaN
3     c       3     c     5.0
4     c       4     c     5.0
5     d       5  NaN     NaN
6     e       6     e     6.0

```

Fig 5.21 (click to enlarge)

Merging on Index

Sometimes, the keys might be the index of a DataFrame. In this case, you can use the argument *left_index=True* or *right_index=True* (or both) to specify that the index will be used as the key to merge.